

College Placement Drive and Student Application Tracker

A web-based system to digitize and streamline campus placement management using Python Flask and SQLite.

GET /api/placements/applications

```
# Presented By

name: ASHIKA.S
department: Information Technology
college: PSVPEC
stack: Python • Flask • SQLite
```

GETTING STARTED

Introduction

Problem Statement

Objective

ARCHITECTURE

System Comparison

Tech Stack

IMPLEMENTATION

Key Features

SQL Queries

Sample Output

File Structure

SUMMARY

How to Run

Future Scope

Conclusion

PROBLEM STATEMENT

Existing System

Placement officers maintain all student data and application statuses manually, leading to errors, delays, and missed opportunities.

01

Manual Record Keeping

Placement records are maintained manually with no standardized format.

02

Scattered Files

Student applications are stored in different files with no central access.

03

No Status Tracking

Difficult to track application status — Applied, Shortlisted, or Rejected.

04

Missed Deadlines

Students miss important placement deadlines due to lack of notifications.

05

No Centralized Database

There is no centralized database for quick search and report generation.

GETTING STARTED

- Introduction
- Problem Statement

Objective

ARCHITECTURE

- System Comparison
- Tech Stack

IMPLEMENTATION

- Key Features
- SQL Queries
- Sample Output
- File Structure

SUMMARY

- How to Run
- Future Scope
- Conclusion

PROJECT GOALS

Objective

Build a centralized, web-based system to manage all placement-related data efficiently and accurately.

01 – Student Data

Store and manage all student information in a structured database.

02 – Company Details

Maintain company records including role, package, and drive date.

03 – Applications

Record and manage student placement applications with applied dates.

04 – Status Tracking

Track application status — Applied, Shortlisted, Interview, Selected, or Rejected.

05 – Search

Search and filter applications by student name or application status.

06 – Reports

Generate placement reports and counts for administrative use.

GETTING STARTED

- Introduction
- Problem Statement
- Objective

ARCHITECTURE

System Comparison

- Tech Stack

IMPLEMENTATION

- Key Features
- SQL Queries
- Sample Output
- File Structure

SUMMARY

- How to Run
- Future Scope
- Conclusion

ARCHITECTURE

Existing vs Proposed System

Moving from a fragmented manual process to a unified, digital, web-based platform.

FEATURE	EXISTING SYSTEM	PROPOSED SYSTEM
Record Keeping	Manual Records	Digital Database
Data Lookup	Difficult Search	Instant Search
Medium	Paper Based	Web Based
Data Integrity	Duplicate Records	Unique Constraints
Processing Speed	Time Consuming	Fast Processing

GETTING STARTED

- Introduction
- Problem Statement
- Objective

ARCHITECTURE

- System Comparison

Tech Stack

IMPLEMENTATION

- Key Features
- SQL Queries
- Sample Output
- File Structure

SUMMARY

- How to Run
- Future Scope
- Conclusion

ARCHITECTURE

Technology Stack

A lightweight, open-source stack — chosen for simplicity, portability, and academic accessibility.

BE

Flask

Backend — Python micro web framework for routing and request handling

DB

SQLite

Database — Serverless relational DB, stores students, companies, applications

FE

HTML & CSS

Frontend — Jinja2 templates with responsive HTML & CSS styling

PY

Python 3.x

Language — Core programming language for all application logic

IDE

VS Code

Development environment with Python and Flask extensions

GETTING STARTED

- Introduction
- Problem Statement
- Objective

ARCHITECTURE

- System Comparison
- Tech Stack

IMPLEMENTATION

Key Features

- SQL Queries
- Sample Output
- File Structure

SUMMARY

- How to Run
- Future Scope
- Conclusion

IMPLEMENTATION

Key Features

All core operations exposed through a simple, responsive web interface.

- Add new placement application
- View all applications
- Search by student name
- Filter by application status
- Track status history
- Record count and statistics
- Simple and responsive web UI

GETTING STARTED

- Introduction
- Problem Statement
- Objective

ARCHITECTURE

- System Comparison
- Tech Stack

IMPLEMENTATION

- Key Features

SQL Queries

- Sample Output
- File Structure

SUMMARY

- How to Run
- Future Scope
- Conclusion

IMPLEMENTATION

SQL Queries (Examples)

Core queries powering the application's data access layer.

Display all applications

```
SELECT * FROM
  Application
ORDER BY applied_date
  DESC;
```

Search by student name

```
SELECT * FROM
  Application
WHERE name LIKE
  '%Kamalika%';
```

Filter shortlisted students

```
SELECT * FROM
  StatusHistory
WHERE status =
  'Shortlisted';
```

Count total applications

```
SELECT
  COUNT(*)
FROM
  Application;
```

GETTING STARTED

- Introduction
- Problem Statement
- Objective

ARCHITECTURE

- System Comparison
- Tech Stack

IMPLEMENTATION

- Key Features
- SQL Queries

Sample Output

- File Structure

SUMMARY

- How to Run
- Future Scope
- Conclusion

IMPLEMENTATION

Sample Output Screens

Four primary screens form the user interface of the Placement Tracker application.

home.html

Home / Dashboard

Landing page with navigation to Add Application, View All, and Search Student.

Add Application

View Applications

Search Student

add_application.html

Add Application

Form to select student and company, set applied date, and submit the application.

POST /api/application/save

applications.html

Application List

Tabular view of all records with filter by status — Applied, Shortlisted, Interview, Rejected.

GET /api/applications

search.html

Search Student

Search by student name and view their company and current placement status.

GET /api/search?name=Kamalika

GETTING STARTED

- Introduction
- Problem Statement
- Objective

ARCHITECTURE

- System Comparison
- Tech Stack

IMPLEMENTATION

- Key Features
- SQL Queries
- Sample Output

File Structure

SUMMARY

- How to Run
- Future Scope
- Conclusion

IMPLEMENTATION

File Structure

Organized project layout for the PlacementTracker application.

```
PlacementTracker/  
  database/  
    schema.sql  
    seed.sql  
  templates/  
    home.html  
    add_application.html  
    applications.html  
    search.html  
  static/  
    style.css  
  app.py  
  requirements.txt  
  README.md
```

- app.py**
Main Flask application — routes, DB connection, request handlers.
- database/**
schema.sql defines all 4 tables; seed.sql inserts sample records for testing.
- templates/**
Jinja2 HTML templates for each view — home, add, list, and search.
- static/style.css**
All UI styling — layout, colors, table, forms, and responsive rules.
- requirements.txt**
Lists Python dependencies — Flask and any additional packages.

GETTING STARTED

- Introduction
- Problem Statement
- Objective

ARCHITECTURE

- System Comparison
- Tech Stack

IMPLEMENTATION

- Key Features
- SQL Queries
- Sample Output
- File Structure

SUMMARY

How to Run

- Future Scope
- Conclusion

SUMMARY

How to Run

Set up and launch the Placement Tracker locally in six steps.

- 1

Install Python 3.x

```
python --version
```
- 2

Install Flask

```
pip install -r requirements.txt
```
- 3

Create Database

```
sqlite3 database.db < database/schema.sql
```
- 4

Insert Sample Data

```
sqlite3 database.db < database/seed.sql
```
- 5

Run the Application

```
python app.py
```
- 6

Open in Browser

```
http://127.0.0.1:5000
```

GETTING STARTED

- Introduction
- Problem Statement
- Objective

ARCHITECTURE

- System Comparison
- Tech Stack

IMPLEMENTATION

- Key Features
- SQL Queries
- Sample Output
- File Structure

SUMMARY

- How to Run
- Future Scope
- Conclusion

SUMMARY

Future Scope

Planned enhancements to elevate the system into a production-grade placement platform.



Admin Login

Role-based access control for placement officers and students.



Email Notifications

Automated email alerts when application status changes.



SMS Alerts

SMS notifications for drive reminders and status updates.



Resume Upload

Allow students to upload and manage their resumes within the portal.



Dashboard Analytics

Visual charts showing placement rates, company-wise stats, and trends.



PDF Reports

Exportable PDF placement reports for college records and accreditation.

GETTING STARTED

- Introduction
- Problem Statement
- Objective

ARCHITECTURE

- System Comparison
- Tech Stack

IMPLEMENTATION

- Key Features
- SQL Queries
- Sample Output
- File Structure

SUMMARY

- How to Run
- Future Scope

Conclusion

SUMMARY

Conclusion

The Placement Tracking System delivers a complete digital solution for campus placement management.

- Successfully developed a Placement Tracking System using Python Flask and SQLite.
- Stores all placement records — students, companies, applications, and status history.
- Reduces manual work and eliminates errors through database constraints and automation.
- Easy to search and filter — by student name, company, or application status.
- Provides accurate placement reports for administrative and accreditation use.

```
return 200 OK // Thank You
```